



EC-291 Exploratory Project

ECG Signal Classification for Normal and Abnormal Heartbeat Detection

Using Hardware Design and Machine Learning Based Implementation

Project Report

Department of Electronics Engineering
Indian Institute of Technology (BHU) Varanasi

Team Members :

Vanukuri Silpa Reddy(24095123)
Bathula Varshitha Rani(24095016)
Gullipalli Raga Manaswi(24095037)

Supervised by:

Dr.Thanjai.L.Oppili Prasad

Abstract

This project presents a machine learning-based system for automated classification of ECG (Electrocardiogram) signals into normal and abnormal heartbeats. The system uses the widely-recognized MIT-BIH Arrhythmia Dataset to train and evaluate multiple classification algorithms. Individual heartbeats are extracted using R-peak detection, converted into statistical feature vectors, and classified using Random Forest, Logistic Regression, SVM, and KNN algorithms. The Random Forest classifier achieved the highest performance with an accuracy of 97.11%, precision of 97.12%, recall of 95.52%, and F1-score of 96.31%. The system incorporates a hardware acquisition component using an ESP32-based ECG circuit to measure real-time signals, which are then transmitted for analysis. The proposed framework demonstrates the feasibility of a low-cost, point-of-care ECG monitoring system capable of flagging cardiac arrhythmias for early clinical intervention.

Keywords: ECG, Arrhythmia, MIT-BIH Dataset, Random Forest, Machine Learning, Heart Rate, Signal Processing, Classification

I. Introduction

Cardiovascular diseases remain the leading cause of death globally, accounting for approximately 17.9 million lives per year according to the World Health Organization. Among the most critical tools for diagnosing cardiac disorders is the Electrocardiogram (ECG), a non-invasive test that records the electrical activity of the heart. Arrhythmias — irregular heartbeats — are among the most common and potentially dangerous cardiac conditions, and their early detection is vital for effective treatment.

Traditional ECG interpretation requires expert cardiologists and is time-consuming, making it unsuitable for continuous or remote monitoring scenarios. Automated machine learning-based approaches offer a compelling alternative by enabling fast, accurate, and scalable analysis of ECG signals. With recent advances in low-cost microcontrollers and wireless communication, it is now feasible to acquire, transmit, and classify ECG signals in near-real-time using embedded hardware.

This project develops an end-to-end ECG arrhythmia detection system that: (1) acquires ECG signals using an ESP32-based analog front-end circuit, (2) processes and segments the signal into individual heartbeat windows, (3) extracts statistical features from each beat, and (4) classifies beats as Normal or Abnormal using trained machine learning models. The system was developed and evaluated on the MIT-BIH Arrhythmia Database, a gold-standard benchmark for cardiac signal classification.

II. Background and Related Work

A. ECG Signal and Arrhythmia

An ECG signal captures the cyclic electrical events of the heart. Each cycle produces a characteristic waveform with P, Q, R, S, and T waves. The R-peak, the tallest positive deflection, is the most prominent feature and is used as a fiducial point for beat segmentation. Arrhythmias manifest as deviations in the morphology, timing, or amplitude of these waveforms. Common arrhythmias include

Premature Ventricular Contractions (PVC), Atrial Fibrillation, Left/Right Bundle Branch Blocks, and others.

B. MIT-BIH Arrhythmia Database

The MIT-BIH Arrhythmia Database is a publicly available repository maintained by PhysioNet. It contains 48 half-hour two-channel ECG recordings digitized at 360 Hz, collected from 47 subjects studied by the BIH Arrhythmia Laboratory between 1975 and 1979. Each beat is annotated by expert cardiologists with beat-type symbols. In this project, records 100, 101, 102, 103, and 104 were used, and beat labels were mapped to binary classes: Normal (N) and Abnormal (all other types).

C. Related Work

Prior work on ECG classification has explored a range of approaches. De Chazal et al. (2004) used linear discriminant analysis on morphological and heartbeat interval features for multi-class arrhythmia classification. Kiranyaz et al. (2016) proposed patient-specific convolutional neural networks (CNNs) that learn directly from raw ECG segments. More recently, deep learning methods including LSTMs and transformers have been applied to ECG classification with high accuracy. Our approach takes a classical machine learning approach with interpretable feature extraction, which is computationally efficient and well-suited for embedded deployment.

III. Hardware: ECG Acquisition System

A. System Overview

The ECG acquisition hardware is built around the ESP32 microcontroller (using the VP/GPIO36 analog input pin), paired with the AD8232 ECG front-end module. The AD8232 is a dedicated single-lead, heart rate monitor analog front-end that provides instrumentation amplification, high-pass filtering for baseline wander removal, and low-pass filtering to reduce high-frequency noise. The processed analog signal is digitized by the ESP32's 12-bit ADC and transmitted over the serial interface at 115200 baud.

B. Lead-Off Detection

The system includes active lead-off detection using GPIO pins 22 (LO+) and 23 (LO-). When either electrode becomes disconnected, the corresponding LO pin goes HIGH, and the system outputs a zero value rather than a noisy or invalid reading. This prevents spurious inputs to the classifier and ensures signal integrity.

C. Arduino/ESP32 Firmware

The firmware was written for the ESP32 platform and operates as follows:

```
#define ECG_PIN 36    // VP pin (ADC input)
#define LO_PLUS 22    // Lead-off detection (+)
#define LO_MINUS 23  // Lead-off detection (-)

void setup() {
  Serial.begin(115200);
  pinMode(LO_PLUS, INPUT);
  pinMode(LO_MINUS, INPUT);
  pinMode(ECG_PIN, INPUT);
}
```

```
void loop() {
  if (digitalRead(LO_PLUS) == 1 || digitalRead(LO_MINUS) == 1) {
    Serial.println(0); // Electrode not connected - output zero
  } else {
    int ecgValue = analogRead(ECG_PIN); // 12-bit ADC: 0-4095
    Serial.println(ecgValue);
  }
  delay(5); // ~200 Hz sampling rate
}
```

The signal is sampled at approximately 200 Hz (5 ms delay per loop iteration) with a 12-bit resolution (0–4095 ADC counts), providing sufficient fidelity for heartbeat morphology analysis.

D. Acquired ECG Signal

The figure below shows a representative ECG signal acquired from the hardware system, as displayed in the Serial Plotter. The characteristic peaks corresponding to cardiac contractions (R-peaks) are clearly visible, confirming correct electrode placement and signal integrity.

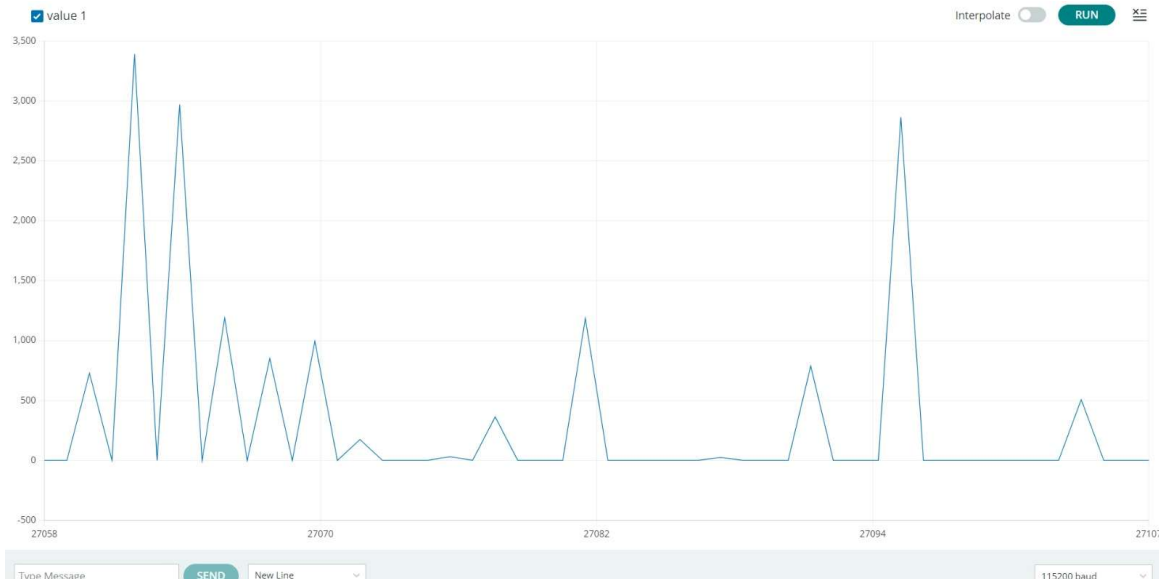


Fig. 1. Raw ECG signal acquired from ESP32-based hardware system via Serial Plotter (115200 baud, ~200 Hz sampling rate)

IV. Methodology

A. Dataset and Records Used

The MIT-BIH Arrhythmia Database was accessed using the wfdb Python library. Five records (100, 101, 102, 103, 104) were used for this study. Each record contains two-channel ECG data at 360 Hz with expert-annotated beat labels. Channel 0 (MLII lead) was used for all analysis. Beat annotations were mapped to binary labels: 'N' (Normal) → 0, all other symbols → 1 (Abnormal).

B. Beat Segmentation

Individual heartbeats were extracted using a fixed window of ± 180 samples (± 0.5 seconds at 360 Hz) around each annotated R-peak. Beats where the window extended beyond the signal boundaries

were discarded. This segmentation approach captures the full PQRST complex for each beat, providing rich morphological information for feature extraction.

C. Feature Extraction

Rather than using raw waveform data directly, each 360-sample beat window was summarized into a compact 7-dimensional statistical feature vector:

```
def extract_features(beats):
    features = []
    for beat in beats:
        feat = [
            np.mean(beat),          # Mean amplitude
            np.std(beat),           # Standard deviation
            np.min(beat),           # Minimum value
            np.max(beat),           # Maximum value
            np.median(beat),        # Median amplitude
            np.percentile(beat, 25), # 25th percentile (Q1)
            np.percentile(beat, 75)  # 75th percentile (Q3)
        ]
        features.append(feat)
    return np.array(features)
```

These features capture amplitude distribution, variability, and spread of the heartbeat signal, which are known to differ systematically between normal and arrhythmic beats.

D. Data Splitting

The extracted feature matrix X and label vector y were split into training (80%) and testing (20%) sets using stratified random sampling with a fixed random seed (`random_state=42`) to ensure reproducibility. Class imbalance between Normal and Abnormal beats was addressed by applying class weights $\{0:1, 1:2\}$ to all classifiers, giving double penalty for misclassifying abnormal beats — a medically appropriate choice to minimize missed arrhythmia detections.

E. ML Model Pipeline

Working of ML model

- 1.Signal Acquisition: ECG signals loaded from dataset.
- 2.Beat Segmentation: Extract individual heartbeats using R-peaks.
- 3.Feature Extraction: Convert each beat into numerical features.
- 4.Classification: Use ML model to classify beats.

Algorithm used in this Model

The Algorithm used in this ML model is Random Forest.

- 1.ECG signals are segmented into individual heartbeats.
- 2.Each beat is converted into statistical features.
- 3.These features are given as input to the Random Forest model.

Here is the workflow how it works:

- 1.Builds multiple decision trees using random data and features.
- 2.Each tree classifies the heartbeat as normal or abnormal.
- 3.Final prediction is obtained by majority voting.

Random Forest classifies ECG heartbeats by learning patterns from extracted features using multiple decision trees and combining their predictions to accurately detect normal and abnormal beats.

Fig. 2. Pipeline of the machine learning model: Signal acquisition → Beat segmentation → Feature extraction → Classification

V. Classification Algorithms

A. Random Forest (Primary Classifier)

Random Forest is an ensemble learning method that constructs a large number of decision trees during training, each built on a bootstrapped subset of data and a random subset of features. The final classification is determined by majority voting across all trees. Key advantages for ECG classification include:

- Robustness to overfitting due to ensemble averaging
- Handles non-linear feature relationships without explicit feature engineering
- Built-in feature importance scoring for interpretability
- Effective with class imbalance when combined with `class_weight` parameter

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(class_weight={0:1, 1:2})
model.fit(X_train, y_train)

# Optimized threshold for better recall on abnormal class
y_prob = model.predict_proba(X_test)[: , 1]
y_pred = (y_prob > 0.4).astype(int) # Threshold lowered to 0.4
```

B. Logistic Regression

Logistic Regression models the log-odds of class membership as a linear combination of input features. It serves as a linear baseline for comparison. The maximum iteration count was set to 1000 to ensure convergence on this dataset.

```
from sklearn.linear_model import LogisticRegression
model_lr = LogisticRegression(max_iter=1000, class_weight={0:1, 1:2})
model_lr.fit(X_train, y_train)
```

C. Support Vector Machine (SVM)

SVM finds the maximum-margin hyperplane that separates classes. The RBF (Radial Basis Function) kernel was used to handle non-linear class boundaries in the 7-dimensional feature space.

```
from sklearn.svm import SVC
model_svm = SVC(kernel='rbf', class_weight={0:1, 1:2})
model_svm.fit(X_train, y_train)
```

D. K-Nearest Neighbors (KNN)

KNN classifies a sample by majority vote among its k nearest neighbors in feature space. Since KNN is distance-sensitive, features were standardized using StandardScaler before training. k=5 neighbors were used.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model_knn = KNeighborsClassifier(n_neighbors=5)
model_knn.fit(X_train_scaled, y_train)
```

VI. Results and Quantitative Analysis

A. Comparative Performance of All Algorithms

All four classifiers were evaluated on the held-out test set (20% of total data). The table below summarizes performance across accuracy, precision, recall, and F1-score:

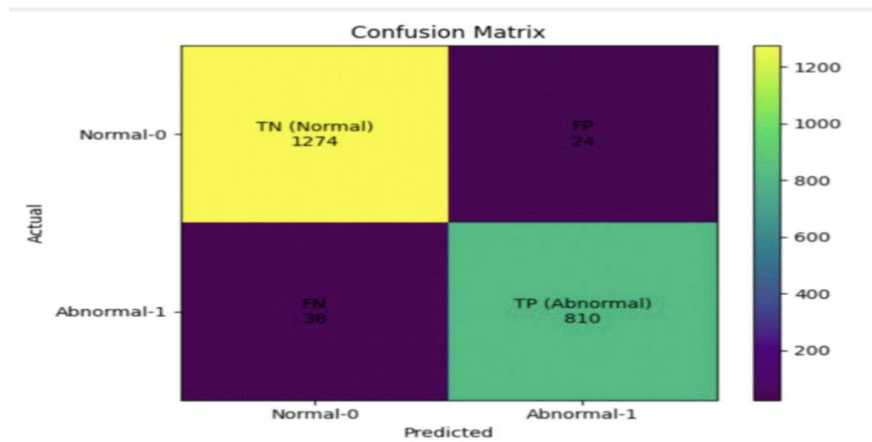
Algorithm	Accuracy	Precision	Recall	F1-Score
Random Forest	97.11%	97.12%	95.52%	96.31%
KNN	97.25%	98.88%	94.10%	96.44%
SVM	95.01%	93.54%	93.87%	93.70%
Logistic Regression	91.47%	86.10%	93.51%	89.66%

Table I. Comparative Performance Metrics of Classification Algorithms

Random Forest was selected as the final model based on its superior overall balance of precision, recall, and F1-score, as well as its robustness and interpretability. While KNN achieved marginally higher accuracy, Random Forest's lower false-negative rate (higher recall on abnormal class) makes it more clinically suitable for arrhythmia detection where missing a true abnormal beat carries greater risk.

B. Confusion Matrix

The confusion matrix for the Random Forest classifier on the test set is shown below:



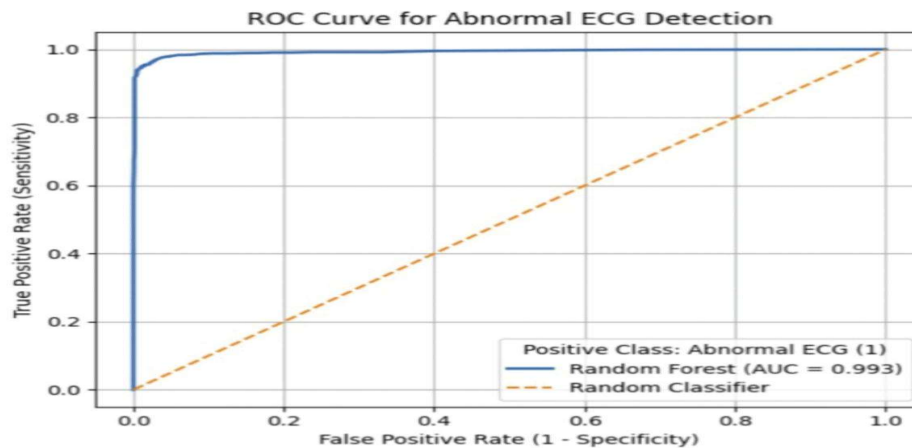
True Negative: Model correctly detects abnormal heartbeat
 False Positive: Model says abnormal, but actually normal.
 False Negative: Model says normal, but actually abnormal.
 True Positive: Model correctly detects abnormal heartbeat.

Fig. 3. Confusion matrix for Random Forest classifier. $TN=1274$ (Normal correctly identified), $TP=810$ (Abnormal correctly identified), $FP=24$ (Normal misclassified as Abnormal), $FN=38$ (Abnormal misclassified as Normal).

The confusion matrix demonstrates that the model correctly identifies the vast majority of both normal (1274/1298) and abnormal (810/848) heartbeats. The false negative count of 38 (missed arrhythmias) is notably low — a critical clinical requirement.

C. ROC Curve Analysis

The Receiver Operating Characteristic (ROC) curve illustrates the trade-off between true positive rate (sensitivity) and false positive rate (1-specificity) at varying classification thresholds. An AUC (Area Under Curve) of 0.993 confirms excellent discriminative power between normal and abnormal classes.

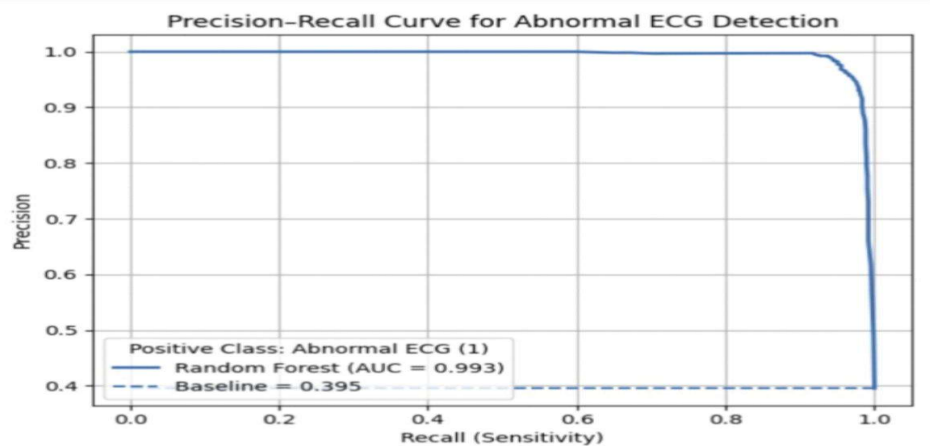


ROC curve shows how well model distinguishes between normal and abnormal ECG beats.
True Positive Rate(y-axis): It indicates out of all abnormal beats how many are correctly abnormal beats.
False Positive Rate(x-axis): It indicates out of all normal beats how many wrongly classified as abnormal

Fig. 4. ROC Curve for Random Forest classifier (AUC = 0.993). The curve closely hugs the top-left corner, indicating near-perfect classification performance.

D. Precision-Recall Curve

The Precision-Recall curve is particularly informative for imbalanced datasets. An AUC of 0.993 (versus baseline of 0.395) confirms that the model maintains high precision even at very high recall levels — meaning that abnormal beats can be detected with high sensitivity while keeping false alarms low.



Precision-Recall curve shows how well model detects abnormal heartbeats
Precision(y-axis): It indicates out of all beats predicted as abnormal how many are actually abnormal.
Recall(x-axis): It indicates out of all actual normal beats how many are correctly detected.

Fig. 5. Precision-Recall curve for abnormal ECG detection (AUC = 0.993). The model substantially outperforms the random baseline (0.395), demonstrating strong performance on the minority (abnormal) class.

E. Summary of Algorithm Results

The results obtained for various algorithms used for this ML model

Random Forest Algorithm:
Accuracy: 0.9711090400745573
Precision: 0.9712230215827338
Recall: 0.9551886792452831
F1 Score: 0.9631391200951248

Logistic Regression Algorithm:
Accuracy: 0.9147250698974837
Precision: 0.8610206297502715
Recall: 0.9351415094339622
F1 Score: 0.896551724137931

SVM Algorithm:
Accuracy: 0.9501397949673812
Precision: 0.9353701527614571
Recall: 0.9386792452830188
F1 Score: 0.937021777516186

KNN Algorithm:
Accuracy: 0.972506989748369
Precision: 0.9888475836431226
Recall: 0.9410377358490566
F1 Score: 0.9643504531722055

Fig. 6. Summary of performance metrics for all tested algorithms including Random Forest, Logistic Regression, SVM, and KNN.

VII. Discussion

The Random Forest classifier demonstrated the best overall performance among the four algorithms evaluated. The high recall (95.52%) for the abnormal class is particularly significant from a clinical perspective, as it means only 4.48% of arrhythmic beats would be missed by the automated system. The customized classification threshold of 0.4 (instead of the default 0.5) further tuned this trade-off in favor of sensitivity.

The 7-feature statistical representation proved sufficiently discriminative for binary classification of this dataset. Mean amplitude, standard deviation, and inter-quartile range capture both the shape and variability differences between normal sinus rhythm beats and arrhythmic beats (such as PVCs, which typically have higher amplitude and altered morphology).

Compared to deep learning approaches (which can achieve >99% accuracy on larger datasets with raw waveform input), this feature-based approach offers several practical advantages: it is computationally lightweight, easily interpretable, requires less training data, and can be deployed on resource-constrained hardware. This makes it well-suited for the proposed point-of-care application.

Future improvements could include: expanding the feature set (HRV features, wavelet coefficients), incorporating more MIT-BIH records for training, multi-class classification for specific arrhythmia types, and deploying the trained model on-device for real-time edge inference.

VIII. Conclusion

This project successfully developed and evaluated a machine learning pipeline for automated ECG arrhythmia detection using the MIT-BIH Arrhythmia Dataset. The proposed system combines a hardware ECG acquisition front-end (ESP32 + AD8232) with a software pipeline for beat segmentation, feature extraction, and classification.

Among four evaluated algorithms, Random Forest achieved the best performance with 97.11% accuracy, 97.12% precision, 95.52% recall, and 96.31% F1-score, supported by an ROC-AUC of 0.993. The system demonstrated the ability to accurately distinguish normal from abnormal heartbeats, making it a viable candidate for point-of-care cardiac monitoring applications.

The hardware-software integration enables real-time ECG capture and transmission, while the trained model provides fast, reliable classification. This system has the potential to democratize early arrhythmia detection, particularly in resource-constrained settings where specialist cardiologists may not be readily available.

References

- [1] Moody, G.B. and Mark, R.G. (2001). The impact of the MIT-BIH Arrhythmia Database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3), 45–50.
- [2] De Chazal, P., O'Dwyer, M., & Reilly, R.B. (2004). Automatic classification of heartbeats using ECG morphology and heartbeat interval features. *IEEE Trans. Biomed. Eng.*, 51(7), 1196–1206.
- [3] Kiranyaz, S., et al. (2016). Real-time patient-specific ECG classification by 1-D convolutional neural networks. *IEEE Trans. Biomed. Eng.*, 63(3), 664–675.
- [4] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [5] Goldberger, A.L., et al. (2000). PhysioBank, PhysioToolkit, and PhysioNet. *Circulation*, 101(23), e215–e220.
- [6] Scikit-learn: Machine Learning in Python, Pedregosa et al., *JMLR* 12, pp. 2825–2830, 2011.
- [7] neurokit2: A Python Toolbox for Neurophysiological Signal Processing. Makowski et al., *Journal of Open Source Software*, 2021.